SUMMER TRAINING PROJECT REPORT



GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

Project Title:
Phishing Website Detection Tool Using Machine Learning

Name: Kunal Kumar Singh
Enrollment No. 14116403224
B.Tech (CSE) Section 3 USICT
Under the Guidance of Mr. Harshit Kumar

Signature

CERTIFICATE

This is to certify that the project report entitled "Phishing Website Detection Tool Using Machine Learning" is a bonafide record of work carried out by Kunal Kumar Singh, Roll Number -14116403224, of the Department of Computer Science and Engineering, Guru Gobind Singh Indraprastha University (GGSIPU) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, during the academic year 2026. The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma.

_____

Project Guide
Mr. Harshit Kumar

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Mr. Harshit kumar, for their valuable guidance, encouragement, and support throughout the course of this project. I am also thankful to the Head of the Department of Computer Science and Engineering and all faculty members for providing the necessary resources and a conducive environment for carrying out this work.

I would also like to thank my family and friends for their constant support and motivation.

Kunal Kumar Singh

Table of Contents

	Page No
1. Introduction	**1**
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope	1
2. Literature Survey	**2**
2.1 Literature Review	2
3. Week 1 Progress Report	**3**
3.1 Objectives of the Week	3
3.2 Progress Achieved	3
3.3 Project Workflow	5
3.4 Tools and Technologies	6
3.5 Skills and Knowledge Acquired	6
3.6 Plan for Week 2	6
4. Week 2 Progress Report	**7**
4.1 Objectives of the Week	7
4.2 Progress Achieved	7
4.3 Model Performance Analysis	11
4.4 Skills and Knowledge Acquired	12
4.5 Plan for Week 3	12
5. Week 3 Progress Report	**13**
5.1 Objectives of the Week	13
5.2 Progress Achieved	13
5.3 Application Architecture	16
5.4 Skills and Knowledge Acquired	16
5.5 Plan for Week 4	17
6. Week 4 Progress Report	**18**
6.1 Objectives of the Week	18
6.2 Progress Achieved	18
6.3 Final Testing and Validation	21
6.4 Skills and Knowledge Acquired	22
6.5 Project Outcome	22
7. Conclusion	**23**
7.1 Conclusion	23
7.2 Future Scope	23
8. References	**24**

CHAPTER 1: INTRODUCTION

1.1 Background

With the increasing use of the internet, phishing attacks have become a major cybersecurity threat. Phishing websites imitate legitimate websites to steal sensitive information such as usernames, passwords, and banking details. Traditional detection methods rely on blacklists, which are ineffective against newly created (zero-day) phishing websites.

Machine Learning (ML) provides a more effective solution by analyzing URL features and learning patterns from known phishing and legitimate websites. This project develops a **Phishing Website Detection Tool Using Machine Learning** that classifies URLs as phishing or legitimate. The trained model is deployed through a **Flask-based web application**, enabling users to verify website URLs in real time.

1.2 Problem Statement

Traditional phishing detection methods rely on blacklists, making them ineffective against newly created phishing websites. Manually identifying phishing websites is also difficult and time-consuming. Therefore, there is a need for an automated machine learning-based system that can accurately classify URLs as legitimate or phishing by analyzing their patterns and characteristics.

1.3 Objectives

The major objectives of this project are:

- Study phishing attacks and understand their impact on cybersecurity.
- Collect and preprocess a publicly available phishing URL dataset.
- Perform exploratory data analysis to understand dataset characteristics.
- Convert textual URLs into numerical feature vectors using CountVectorizer.
- Train multiple machine learning models for phishing URL classification.
- Compare model performance using standard evaluation metrics.
- Select the best-performing model for deployment.
- Develop a Flask-based web application for real-time phishing website detection.

1.4 Scope

This project focuses on detecting phishing websites using machine learning based on URL analysis. The system accepts a user-provided URL and predicts whether it is legitimate or phishing. It is limited to lexical URL analysis and does not examine webpage content, SSL certificates, or DNS records. The model is deployed as a Flask-based web application to demonstrate real-time phishing detection for educational purposes.

CHAPTER 2: LITERATURE SURVEY

2.1 Literature Review

Phishing website detection has become an important area of cybersecurity research due to the increasing number of online attacks targeting internet users. Early detection methods mainly relied on blacklist-based techniques, where URLs were compared against databases of known phishing websites. Although these methods are effective for identifying previously reported phishing websites, they cannot detect newly created or zero-day phishing attacks, making them less effective against evolving threats [7].

To address these limitations, researchers introduced machine learning techniques that classify websites based on patterns learned from previously labeled phishing and legitimate URLs. Unlike traditional approaches, machine learning models can identify suspicious URLs without relying solely on predefined rules or blacklists. Common supervised learning algorithms such as Logistic Regression, Decision Trees, Random Forests, Naïve Bayes, and Support Vector Machines have been widely used because they provide high accuracy and good generalization for phishing URL classification [5], [6], [10].

Since URLs are textual data, feature extraction plays a crucial role in phishing detection. Techniques such as CountVectorizer and TF-IDF transform URLs into numerical feature vectors, enabling machine learning algorithms to process them effectively. These methods capture important word and character patterns that help distinguish phishing URLs from legitimate ones [2], [9].

In this project, the **Phishing Site URLs Dataset** from Kaggle is used to train and evaluate multiple machine learning models [1]. The implementation is carried out using **Python** and the **Scikit-learn** library for data preprocessing, feature extraction, model training, and evaluation [2], [3]. The best-performing model is then deployed through a **Flask** web application, allowing users to verify website URLs in real time [4]. This approach provides an efficient and practical solution for phishing website detection by combining machine learning techniques with an easy-to-use web interface. Additionally, the developed system is designed to provide fast predictions with minimal computational overhead, making it suitable for real-time use. The comparison of multiple classification algorithms also helps identify the most accurate and reliable model for phishing URL detection, improving the overall effectiveness of the proposed solution.

Chapter 3 : WEEK 1 PROGRESS REPORT

3.1 Objectives of the Week

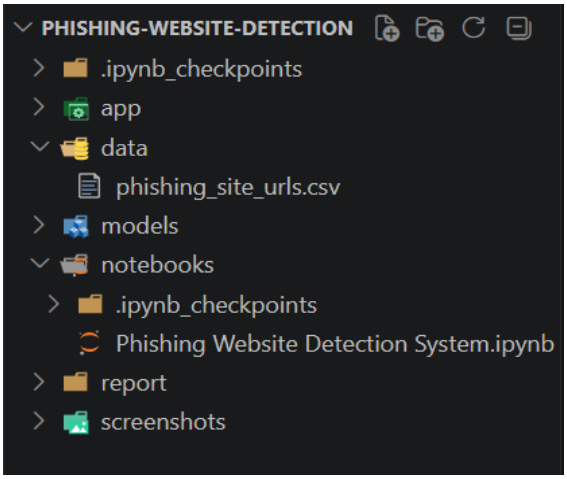
The primary objectives planned for the first week of the project were:

- Finalize the project approach and define its scope.
- Understand phishing websites and their impact on cybersecurity.
- Study the role of machine learning in phishing website detection.
- Select an appropriate publicly available dataset.
- Set up the development environment.
- Plan the implementation methodology for the project.
- Begin the initial implementation of the project.

3.2 Progress Achieved

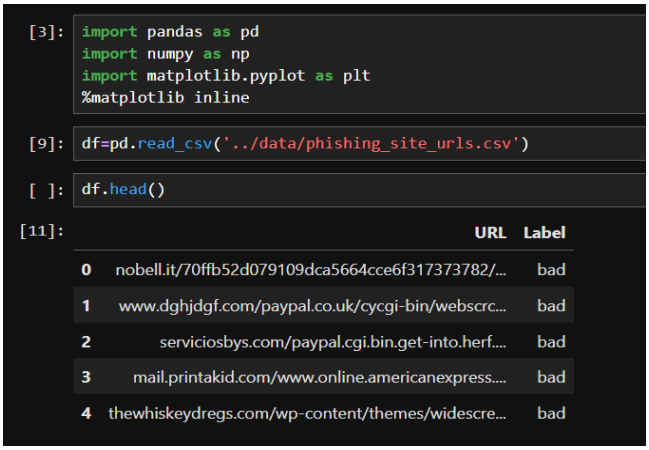
During the first week, the project planning and initial development activities were completed. The focus was on understanding the problem domain, selecting an appropriate dataset, setting up the development environment, and initiating the implementation process.

- Studied the fundamentals of phishing attacks, phishing websites, and their impact on users through an extensive review of research papers and other credible technical resources.
- Finalized the project approach **"Phishing Website Detection Tool Using Machine Learning."**
- Selected the **Phishing Site URLs** dataset from **Kaggle** for model development and initial analysis.
    - Created the project directory structure and initialized the development environment.



- Loaded the dataset into Python and performed an initial inspection of the dataset.

Figure 1. Initial preview of the phishing website dataset using the head() function.



```
[3]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     %matplotlib inline

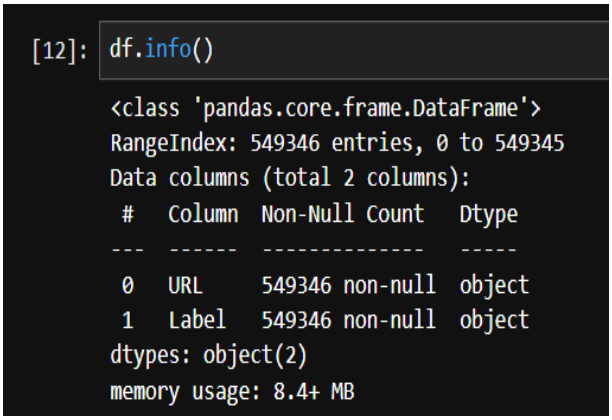
[9]: df=pd.read_csv('../data/phishing_site_urls.csv')

[ ]: df.head()
```

[11]:

	URL	Label
0	nobell.it/70ffb52d079109dca5664cce6f317373782/...	bad
1	www.dghjdgf.com/paypal.co.uk/cycgi-bin/webscrc...	bad
2	serviciosbys.com/paypal.cgi.bin.get-into.herf....	bad
3	mail.printakid.com/www.online.americanexpress....	bad
4	thewhiskeydregs.com/wp-content/themes/widescre...	bad

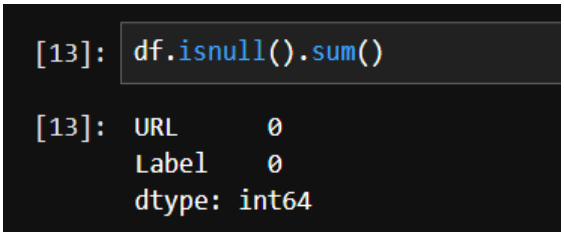
Figure 2. Dataset information displaying the number of records, features, and data types.



```
[12]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 549346 entries, 0 to 549345
Data columns (total 2 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   URL     549346 non-null  object
 1   Label   549346 non-null  object
dtypes: object(2)
memory usage: 8.4+ MB
```

Figure 3. Missing value analysis of the dataset.



```
[13]: df.isnull().sum()

[13]: URL      0
      Label    0
      dtype: int64
```

Figure 4. Distribution of phishing and legitimate URLs in the dataset.

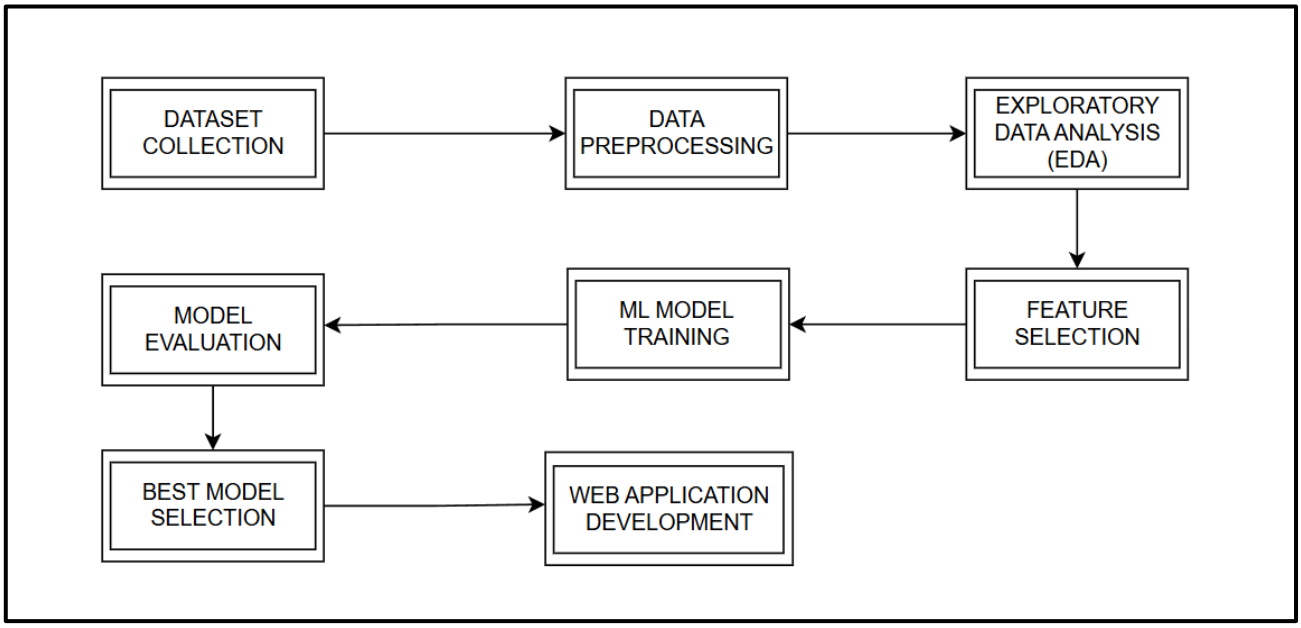
```
[17]: df.Label.value_counts()

[17]: Label
      good    392924
      bad     156422
      Name: count, dtype: int64
```

- Started preparing the project documentation and implementation plan.

3.3 Project Workflow

The development of the phishing website detection system follows a structured machine learning pipeline. The workflow begins with dataset collection, followed by preprocessing, exploratory data analysis, feature engineering, model training, evaluation, and finally deployment through a web application.



3.4 Tools and Technologies

Tool/Technology	Purpose
Python	Programming language used for implementation
Pandas	Data loading and preprocessing
NumPy	Numerical computations
Matplotlib	Data visualization
Seaborn	Exploratory data analysis and statistical plots
Scikit-learn	Machine learning model development and evaluation
Flask	Web application development
Joblib	Model serialization and loading
Visual Studio Code / Jupyter Notebook	Development environment
Git & GitHub	Version control and project management

3.5 Skills and Knowledge Acquired

During the first week, the following concepts were learned:

- Fundamentals of phishing attacks.
- Overview of machine learning for cybersecurity.
- Dataset handling using Python.
- Python environment configuration.
- Project planning and workflow design.
- Basic understanding of phishing website datasets.

3.6 Plan for Week 2

The following tasks are planned for the second week:

- Perform data preprocessing.
- Conduct exploratory data analysis (EDA).
- Train multiple machine learning models.
- Compare model performance using evaluation metrics.
- Select the best-performing model for deployment.

Chapter 4 : WEEK 2 PROGRESS REPORT

4.1 Objectives of the Week

The primary objectives planned for the second week were:

- Perform data preprocessing and cleaning.
- Conduct Exploratory Data Analysis (EDA).
- Convert textual URL data into numerical format suitable for machine learning.
- Train multiple machine learning models.
- Evaluate and compare model performance.
- Select the most suitable model for deployment.

4.2 Progress Achieved

During the second week, the project moved from the planning phase to the implementation phase. The major activities included data preprocessing, exploratory data analysis, text vectorization, machine learning model development, and performance evaluation. These steps prepared the phishing URL dataset for effective classification and helped identify the most suitable model for deployment.

The dataset was initially examined to understand its structure, verify data consistency, and identify any missing or duplicate records. The inspection confirmed that the dataset contained the expected columns and was suitable for further processing. Missing value analysis showed that no null values were present, while duplicate URLs, if any, were removed to avoid redundancy during model training.

The phishing labels were then converted into numerical values, where **Good URLs were encoded as 0** and **Phishing URLs were encoded as 1**, making the dataset compatible with supervised machine learning algorithms.

Figure 5. Dataset preview after label encoding.

```
[30]: df['Label']=df['Label'].map({
          'good':0,
          'bad':1
      })

[31]: df.head()
```

[31]:

	URL	Label
0	nobell.it/70ffb52d079109dca5664cce6f317373782/...	1
1	www.dghjdgf.com/paypal.co.uk/cycgi-bin/webscrc...	1
2	serviciosbys.com/paypal.cgi.bin.get-into.herf....	1
3	mail.printakid.com/www.online.americanexpress....	1
4	thewhiskeydregs.com/wp-content/themes/widescre...	1

The above figure confirms that the categorical labels have been successfully transformed into numerical values while preserving the original URL information.

To better understand the characteristics of phishing websites, several lexical features were extracted from each URL, including URL length, number of dots, digits, hyphens, slashes, HTTPS presence, IP address occurrence, and special characters. These features were primarily used for exploratory analysis and understanding the behaviour of phishing URLs.

Figure 6. Dataset after extracting lexical URL features.

```
[33]: df['URL_Length'] = df['URL'].apply(len)
      df['Dots'] = df['URL'].str.count(r'\.')
      df['Hyphens'] = df['URL'].str.count('-')
      df['Digits'] = df['URL'].str.count(r'\d')
      df['Slashes'] = df['URL'].str.count('/')
      df['HTTPS'] = df['URL'].str.contains('https').astype(int)
      df['IP_Address'] = df['URL'].str.contains(
          r'\b(?:\d{1,3}\.){3}\d{1,3}\b',
          regex=True
      ).astype(int)
      df['Special_Chars'] = df['URL'].str.count(r'[@?=&_%]')

[34]: df.head()
```

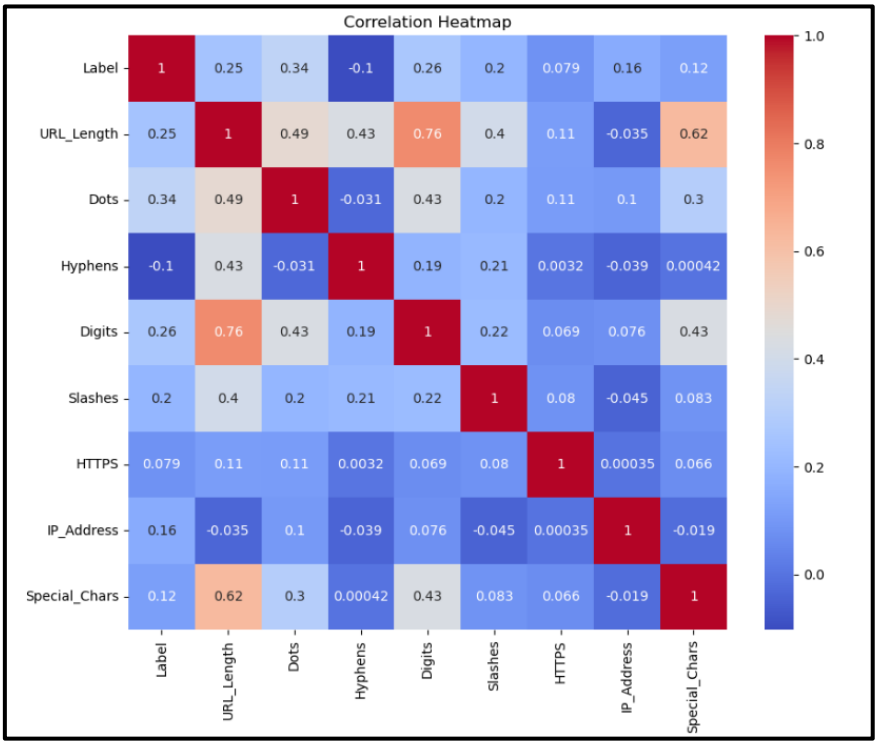
[34]:

	URL	Label	URL_Length	Dots	Hyphens	Digits	Slashes	HTTPS	IP_Address	Special_Chars
0	nobell.it/70ffb52d079109dca5664cce6f317373782/...	1	225	6	4	58	10	0	0	12
1	www.dghjdgf.com/paypal.co.uk/cycgi-bin/webscrc...	1	81	5	2	1	4	0	0	4
2	serviciosbys.com/paypal.cgi.bin.get-into.herf....	1	177	7	1	47	11	0	0	0
3	mail.printakid.com/www.online.americanexpress....	1	60	6	0	0	2	0	0	0
4	thewhiskeydregs.com/wp-content/themes/widescre...	1	116	1	1	21	10	0	0	1

The extracted features provide valuable insights into URL patterns and assist in understanding the differences between legitimate and phishing websites.

Exploratory Data Analysis (EDA) was carried out to study the relationships among the extracted features. A correlation heatmap was generated to identify how different URL characteristics influence each other.

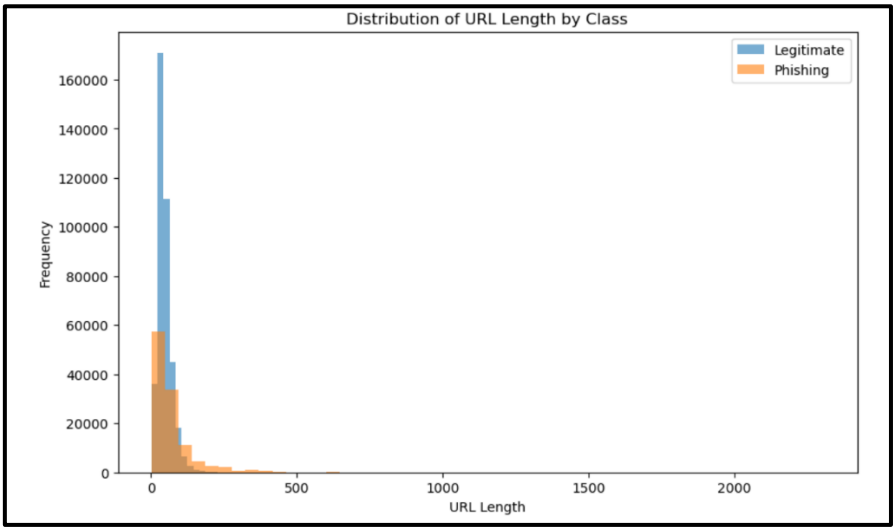
Figure 7. Correlation heatmap of extracted URL features.



The heatmap indicates that some lexical features exhibit moderate correlation, while most features remain relatively independent. This suggests that multiple URL characteristics collectively contribute to phishing detection.

The distribution of URL lengths was also analyzed using a histogram. This visualization helps understand whether phishing websites tend to have unusually short or long URLs.

Figure 8. Distribution of URL lengths.



The figure compares the URL length distributions of legitimate and phishing websites. It can be observed that phishing URLs generally exhibit greater variability and tend to be longer than many legitimate URLs, although there is some overlap between the two classes. This indicates that URL length alone is not sufficient for classification but serves as a useful feature when combined with other URL characteristics.

Since machine learning algorithms cannot process raw text directly, the URL strings were transformed into numerical feature vectors using the **CountVectorizer** technique with character-level n-grams. This conversion enabled the algorithms to learn meaningful textual patterns present within phishing and legitimate URLs.

Figure 9. Shape of the CountVectorizer transformed dataset.

```
[41]: X_train_vec=vectorizer.fit_transform(X_train)

      X_test_vec=vectorizer.transform(X_test)

[42]: print(X_train_vec.shape)

      print(X_test_vec.shape)

      (405756, 100000)
      (101440, 100000)
```

The transformed dataset contains high-dimensional sparse feature vectors representing the textual structure of each URL, making it suitable for machine learning model training.

After vectorization, the dataset was divided into training and testing subsets. Three supervised machine learning algorithms—**Logistic Regression**, **Multinomial Naïve Bayes**, and **Linear Support Vector Machine (Linear SVM)**—were trained and evaluated using the transformed dataset.

The performance of each classifier was measured using standard evaluation metrics, including Accuracy, Precision, Recall, and F1-Score.

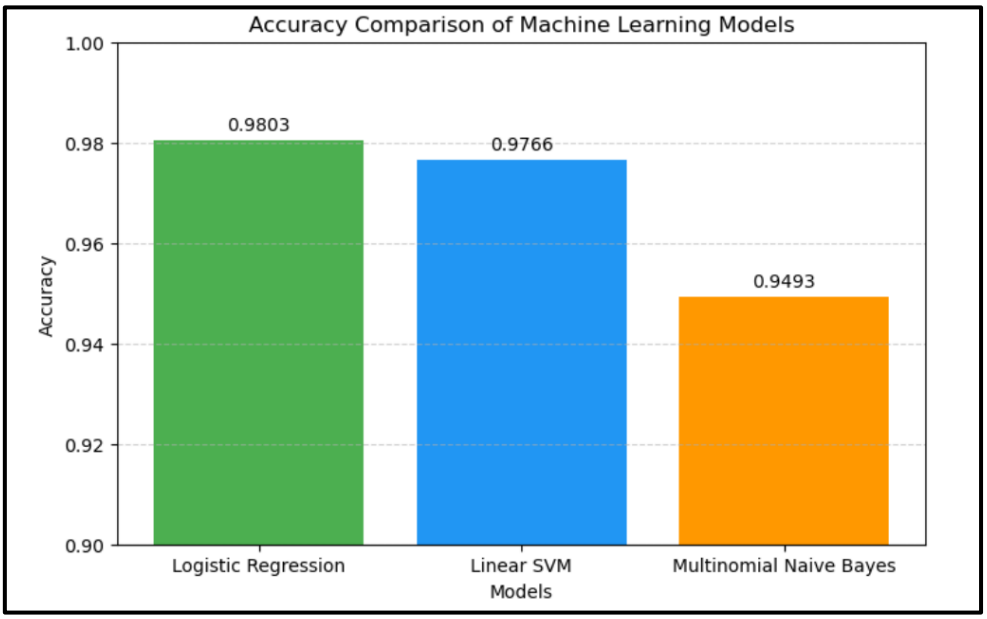
Figure 10. Performance comparison of machine learning models.

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.980323	0.966589	0.945363	0.955858
2	Linear SVM	0.976577	0.951547	0.944138	0.947828
1	Multinomial Naive Bayes	0.949290	0.912000	0.857743	0.884040

The comparison table presents the performance of all trained classifiers and provides the basis for selecting the most suitable model for deployment.

To provide a clearer comparison, the obtained accuracies were also represented graphically.

Figure 11. Accuracy comparison of trained machine learning models.



The graphical comparison makes it easier to visualize the relative performance of each classifier and highlights the best-performing model.

4.3 Model Performance Analysis

The trained machine learning models were compared based on their classification accuracy and overall predictive performance. Among the evaluated algorithms, **Logistic Regression** achieved the highest accuracy while maintaining balanced precision and recall values. The remaining models also performed satisfactorily; however, slight differences in accuracy and generalization capability were observed.

Based on these evaluation results, the **Logistic Regression** was selected as the final model for deployment because it demonstrated the most reliable performance on unseen data. The selected model was further analyzed using a Classification Report and Confusion Matrix.

Figure 12. Classification report of the selected machine learning model.

```
[54]: best_model = models[best_model_name]

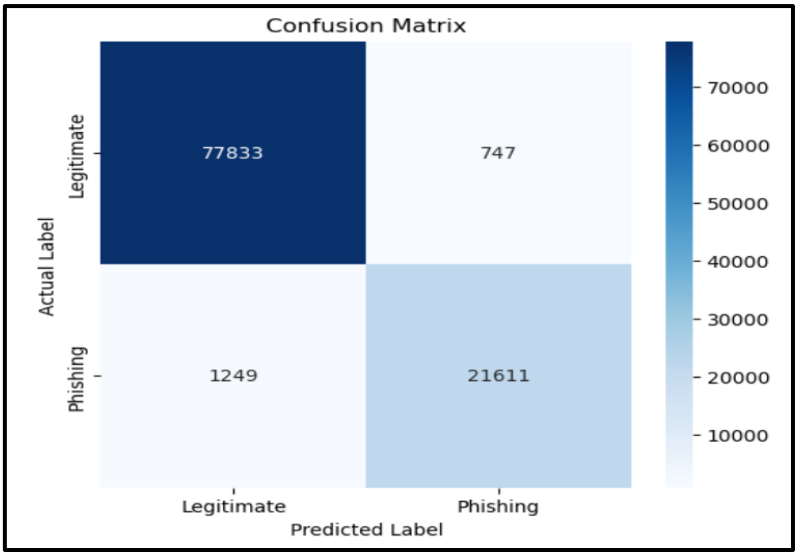
[55]: y_pred = best_model.predict(X_test_vec)

[56]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.98	0.99	0.99	78580
1	0.97	0.95	0.96	22860
accuracy			0.98	101440
macro avg	0.98	0.97	0.97	101440
weighted avg	0.98	0.98	0.98	101440

The classification report summarizes the Precision, Recall, F1-Score, and overall Accuracy achieved by the selected classifier.

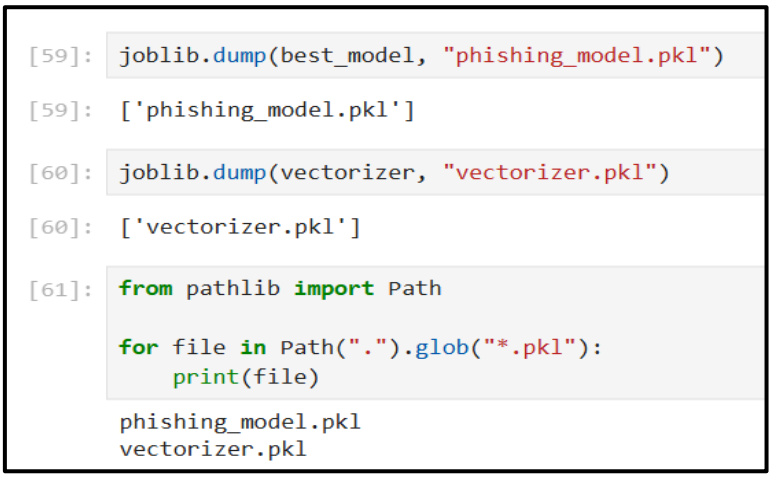
Figure 13. Confusion matrix of the selected machine learning model.



The confusion matrix illustrates the number of correctly and incorrectly classified phishing and legitimate URLs, providing a comprehensive evaluation of the classifier's prediction capability.

To prepare the model for deployment, the trained machine learning model and the fitted CountVectorizer were serialized using the **Joblib** library. Saving these objects ensures that the model can be loaded directly into the Flask application during deployment without retraining.

Figure 14. Saving the trained model and CountVectorizer using Joblib.



Model serialization marks the completion of the machine learning phase and prepares the project for web application integration during the final week.

4.4 Skills and Knowledge Acquired

During the second week, the following concepts and practical skills were developed:

- Data preprocessing and cleaning techniques.
- Label encoding for supervised learning.
- URL feature extraction and lexical analysis.
- Exploratory Data Analysis (EDA).
- Text vectorization using CountVectorizer.
- Machine learning model development using Scikit-learn.
- Performance evaluation using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- Model comparison and selection.
- Model serialization using Joblib for deployment.

4.5 Plan for Week 3

- Develop the Flask web application.
- Load the trained Logistic Regression model and CountVectorizer using Joblib.
- Integrate the machine learning model with the Flask application.
- Design a simple and user-friendly interface for URL prediction.
- Perform initial testing of the application using sample URLs.

CHAPTER 5: WEEK 3 PROGRESS REPORT

5.1 Objectives of the Week

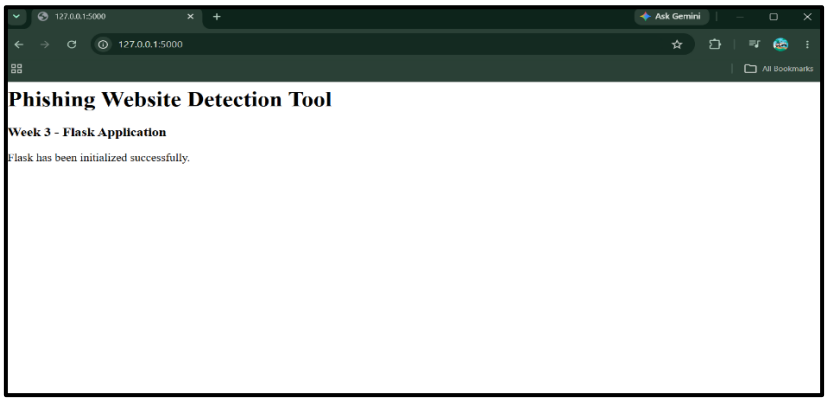
- The primary objectives planned for the third week were:
- Develop the web application using Flask.
- Load the trained machine learning model and CountVectorizer using Joblib.
- Integrate the prediction model with the Flask application.
- Design a simple and user-friendly interface for URL prediction.
- Perform initial testing of the application using sample URLs.

5.2 Progress Achieved

During the third week, the machine learning model developed in the previous phase was successfully integrated into a Flask-based web application. The objective of this phase was to transform the trained classifier into an interactive application capable of performing real-time phishing URL detection.

The Flask framework was successfully initialized by creating a basic web application and running it on a local development server. This confirmed that the application environment was properly configured and ready for integrating the trained machine learning model.

Figure 15. Flask application initialized and running successfully.



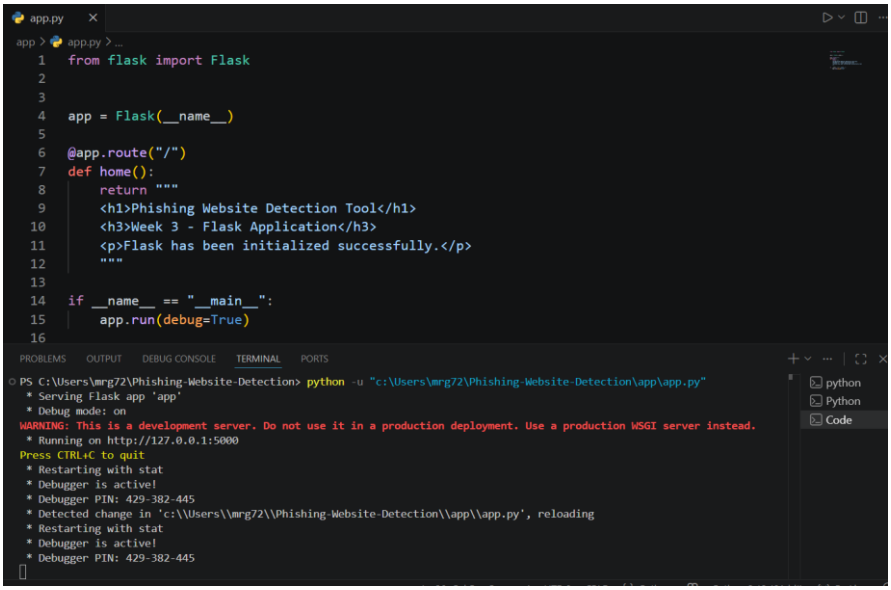


Figure 16. Loading the trained model and CountVectorizer using Joblib.

app.py

```python
from flask import Flask
import joblib
import os

app = Flask(__name__)
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

MODEL_PATH = os.path.join(BASE_DIR, "..", "models", "phishing_model.pkl")
VECTORIZER_PATH = os.path.join(BASE_DIR, "..", "models", "vectorizer.pkl")

try:
    model = joblib.load(MODEL_PATH)
    vectorizer = joblib.load(VECTORIZER_PATH)

    print("✅ Machine Learning Model Loaded Successfully!")
    print("✅ CountVectorizer Loaded Successfully!")

except Exception as e:
    print("❌ Error Loading Files")
    print(e)


@app.route("/")
def home():
    return """
    <h1>Phishing Website Detection Tool</h1>
    <h3>Week 3 - Flask Application</h3>
    <p>Machine Learning Model Loaded Successfully.</p>
    """


if __name__ == "__main__":
    app.run(debug=True)
```

A prediction function was implemented to process user-provided URLs. Whenever a URL is submitted through the web interface, it is first transformed into numerical feature vectors using the loaded CountVectorizer. The transformed data is then passed to the trained Logistic Regression model, which predicts whether the entered URL is **Legitimate** or **Phishing**.

Figure 17. URL prediction function implemented in Flask.

```python
def predict_url(url):
    """
    Predict whether a URL is Legitimate or Phishing.
    """

    url_vector = vectorizer.transform([url])

    prediction = model.predict(url_vector)[0]


    if prediction == 0:
        return "Legitimate"
    else:
        return "Phishing"
```

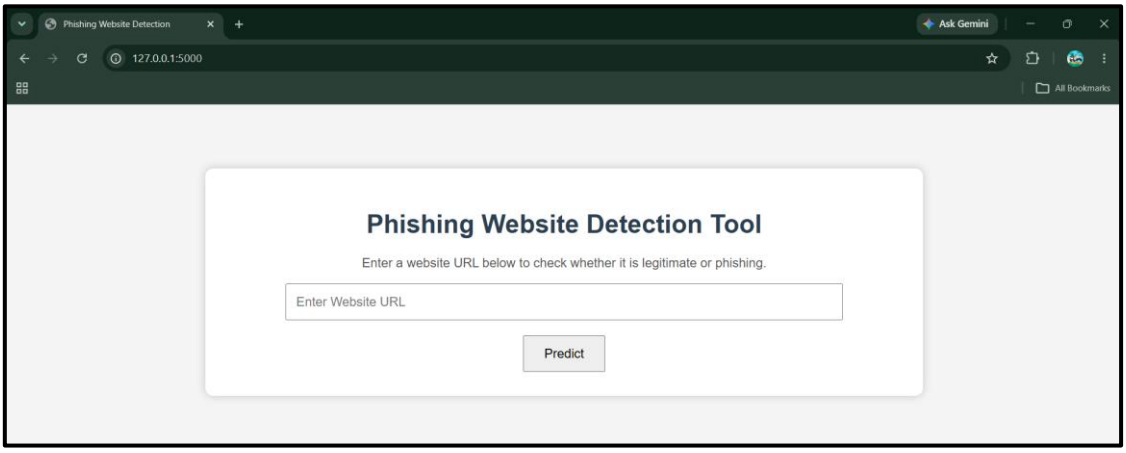
OUTPUT

```
✅ Machine Learning Model Loaded Successfully!
✅ CountVectorizer Loaded Successfully!

----------------------------------------
 Initial Testing of Prediction Function
----------------------------------------
URL        : github.com/ReactORM/
Prediction : Legitimate Website
----------------------------------------
URL        : github.com/git-guide/
Prediction : Legitimate Website
----------------------------------------
URL        : math.unm.edu/ACA/2001/2001.html
Prediction : Legitimate Website
----------------------------------------
URL        : amazon.de-register.net/
Prediction : Phishing Website
----------------------------------------
URL        : supported-amazon.net/
Prediction : Legitimate Website
----------------------------------------
URL        : docs.google.com/spreadsheet/viewform?formkey=test
Prediction : Phishing Website
----------------------------------------
 * Debugger is active!
 * Debugger PIN: 429-382-445
```

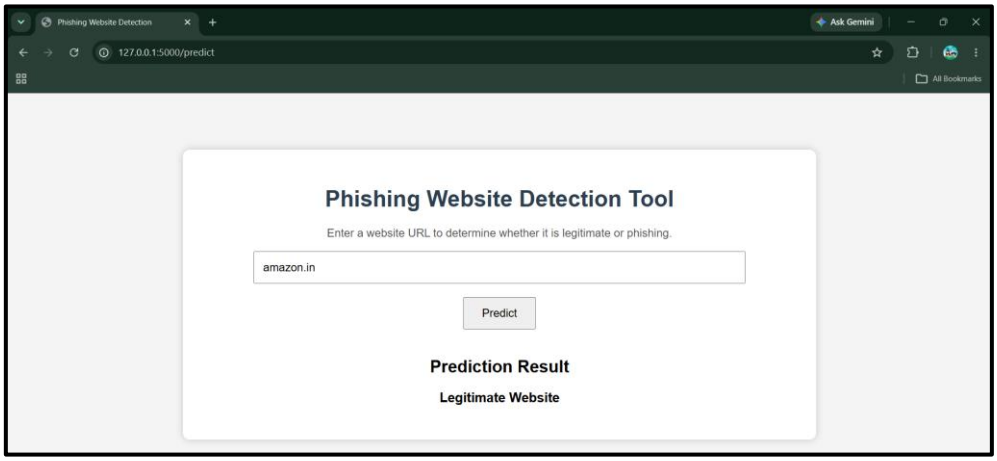
A simple and user-friendly web interface was developed using HTML and CSS. The interface allows users to enter a website URL through an input field and obtain the prediction result with a single button click. The application displays whether the entered URL is classified as legitimate or phishing.

Figure 18. Home page of the phishing website detection application.



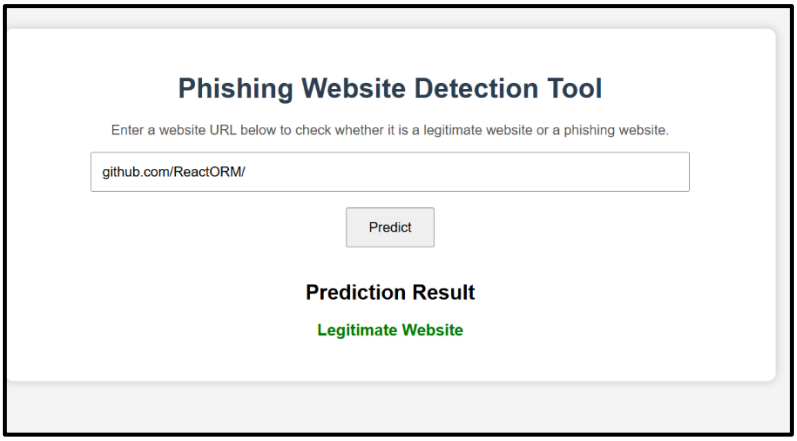
The prediction result page was designed to clearly present the classification returned by the model. The output is displayed in an easily understandable format, allowing users to quickly determine whether a website appears to be safe or potentially malicious.

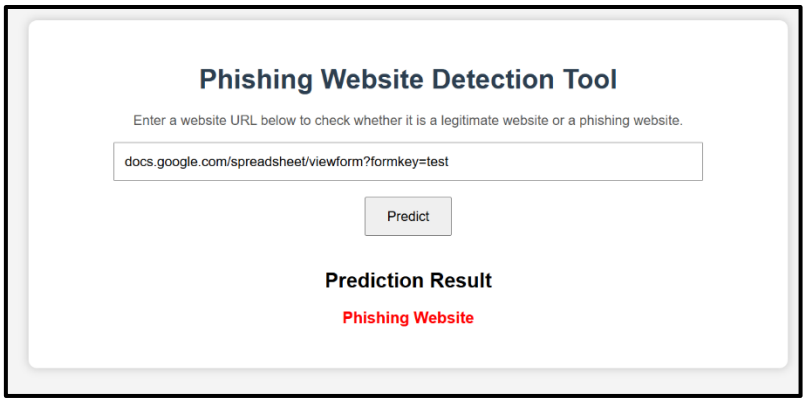
Figure 19. Prediction result displayed by the application.



Initial functional testing was performed using a small collection of legitimate and phishing URLs to verify that the application correctly loads the machine learning model, processes user input, and produces predictions without runtime errors.

Figure 20. Initial testing of the web application using sample URLs.



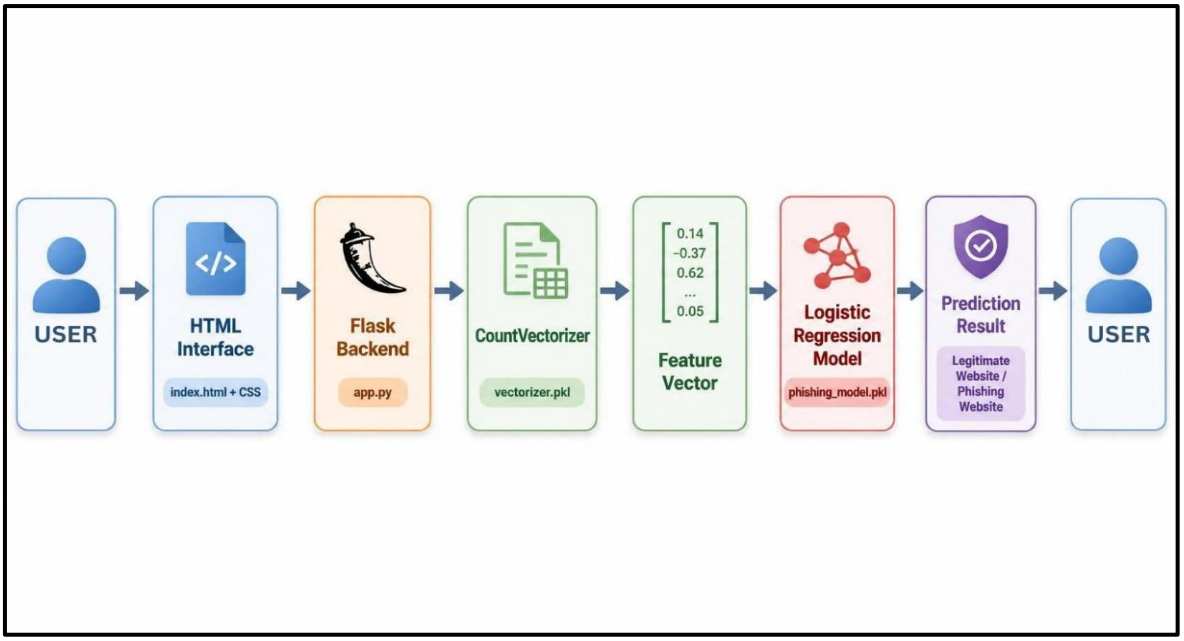


The successful integration of the trained machine learning model with the Flask framework completed the application development phase and prepared the project for comprehensive testing and optimization during the final week.

5.3 Application Architecture

- The deployed application follows a simple client-server architecture.
- The user enters a URL through the web interface.
- Flask receives the request.
- The CountVectorizer converts the URL into numerical feature vectors.
- The Logistic Regression model predicts the URL category.
- Flask returns the prediction result to the user interface.

Figure 21. Architecture of the Flask-based Phishing Website Detection System



5.4 Skills and Knowledge Acquired

- During the third week, the following concepts and practical skills were developed:
- Flask web framework development.
- Model loading using Joblib.
- Integration of machine learning models with web applications.
- HTML form handling in Flask.
- Routing and request processing.
- User interface design using HTML and CSS.
- Implementation of real-time URL prediction.
- Basic application testing and debugging.

5.5 Plan for Week 4

The following tasks are planned for the fourth and final week of the project:

- Perform comprehensive testing using both phishing and legitimate URLs.
- Improve the user interface and overall user experience.
- Handle invalid URL inputs and implement error handling.
- Optimize application performance and remove identified bugs.
- Validate prediction results using additional test cases.
- Capture application screenshots for project documentation.
- Prepare the final project report.

CHAPTER 6: WEEK 4 PROGRESS REPORT

6.1 Objectives of the Week

The primary objectives planned for the fourth week were:

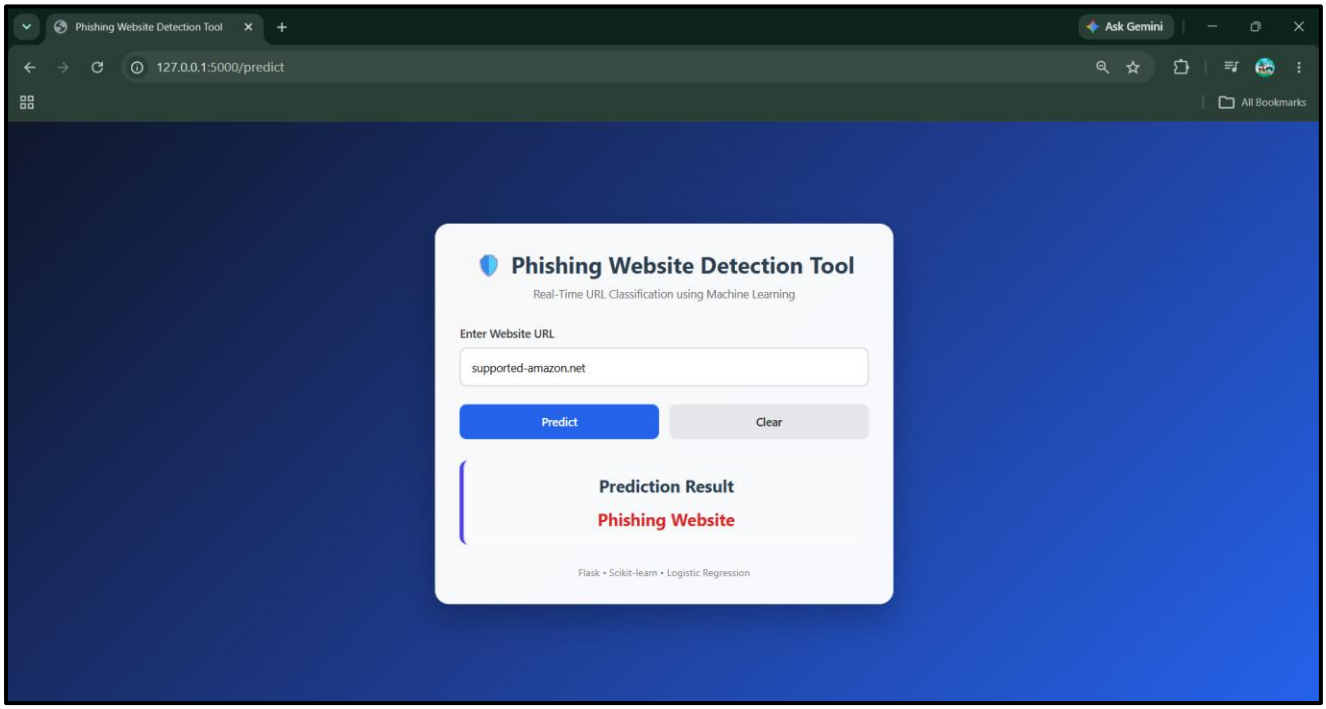
- Perform comprehensive testing using both legitimate and phishing URLs.
- Improve the user interface and overall user experience.
- Implement input validation and basic error handling.
- Optimize the application and fix identified issues.
- Validate prediction results using additional test cases.
- Capture screenshots for project documentation.
- Complete the final project report.

6.2 Progress Achieved

During the fourth and final week, the phishing website detection application was refined through comprehensive testing, interface improvements, and application optimization. The primary objective of this phase was to enhance the usability, reliability, and overall presentation of the deployed system while ensuring that the application operated correctly under different testing scenarios.

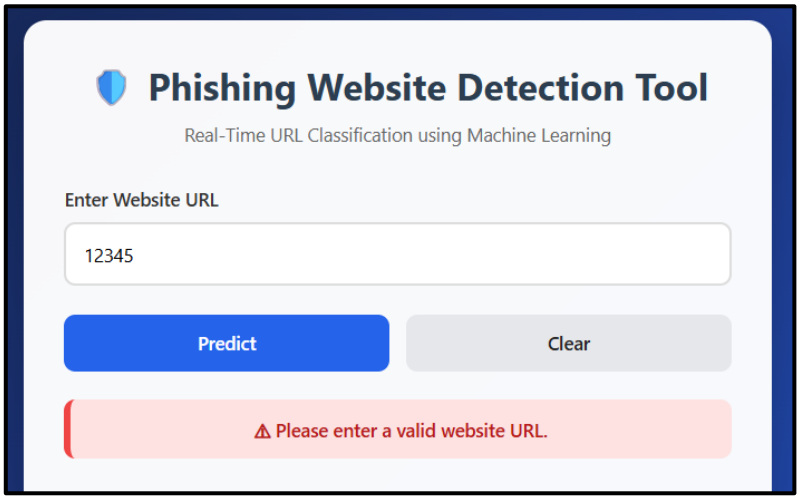
The web interface was improved to provide a cleaner and more user-friendly experience. Minor modifications were made to the layout, styling, and prediction result display to improve readability and make the application easier to use. The prediction results were displayed using color-coded messages, enabling users to distinguish between legitimate and phishing websites more easily.

Figure 22. Improved user interface of the phishing website detection application.



Basic input validation was implemented to ensure that empty or invalid inputs were handled appropriately before prediction. This prevented unnecessary processing and improved the robustness of the application by providing meaningful feedback whenever invalid input was entered.

Figure 23. Input validation and error handling implemented in the application.



Comprehensive functional testing was performed using a larger collection of legitimate and phishing URLs. Different categories of URLs were submitted through the web interface to verify that the application correctly processed user input, transformed URLs using the saved CountVectorizer, and generated predictions using the trained Logistic Regression model. The testing confirmed that the complete prediction pipeline operated successfully without runtime errors.

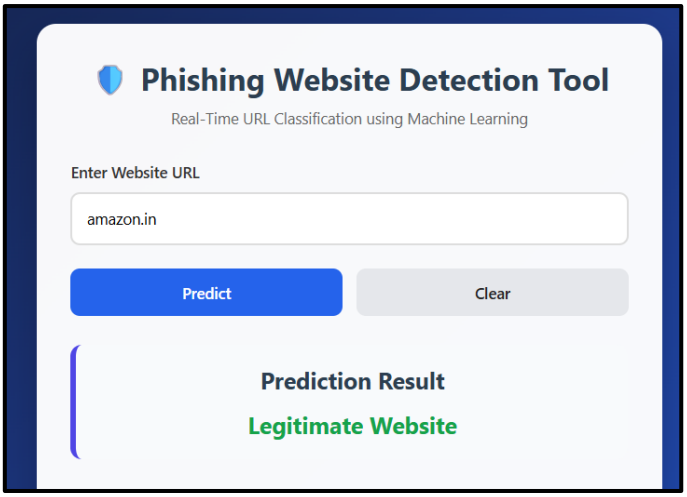
Legitimate URLs

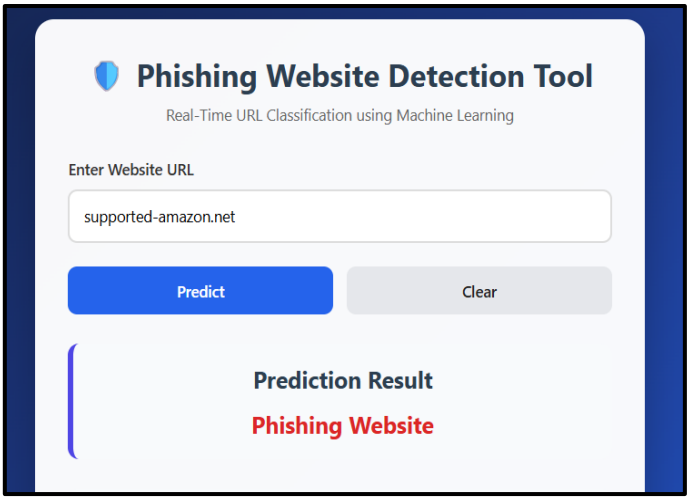
URL	Expected Result
www.reddit.com/r/cpp/	Legitimate
amazon.in	Legitimate
wikipedia.org	Legitimate
python.org	Legitimate
www.microsoft.com	Legitimate

Phishing URLs

URL	Expected Result
supported-amazon.net	Phishing
amazon.de-register.net	Phishing
docs.google.com/spreadsheet/viewform?formkey=test	Phishing
paypal-login-secure.net	Phishing
facebook-security-login.xyz	Phishing

Figure 24. Comprehensive testing of the application using multiple sample URLs.

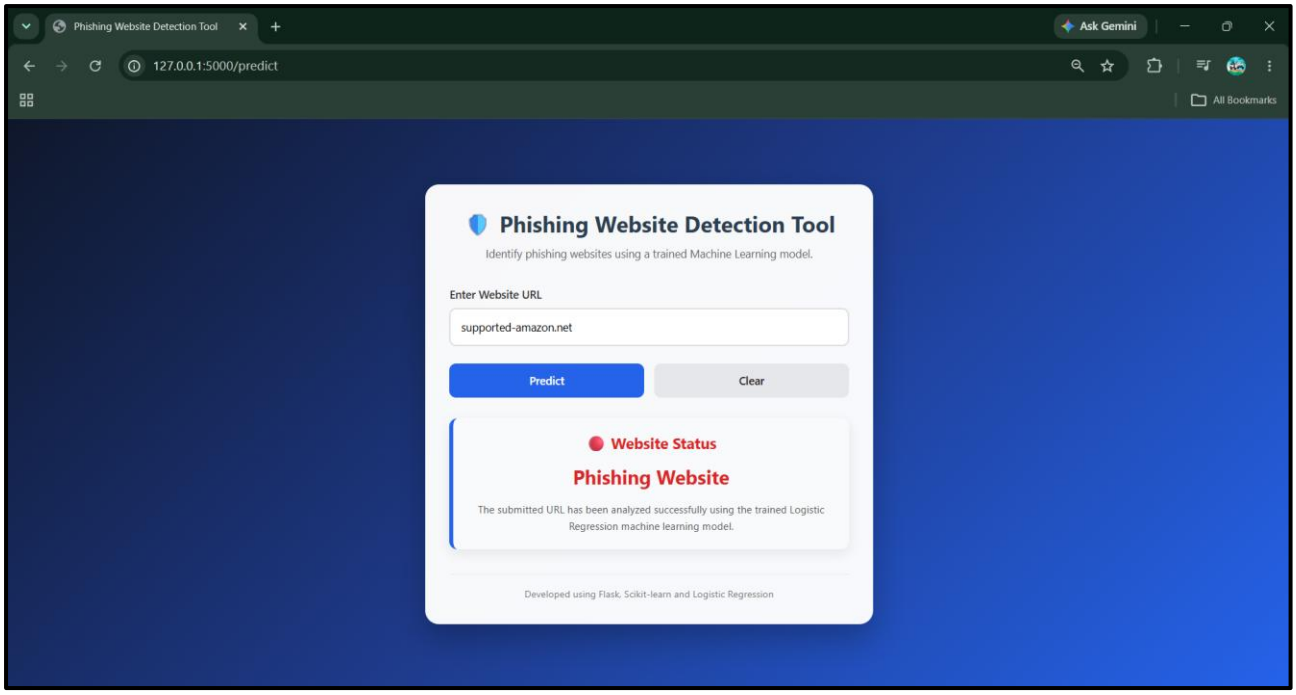




Minor code optimizations and debugging activities were carried out to improve application stability and maintainability. The routing logic, prediction workflow, and overall project organization were reviewed to ensure smooth execution. The final application successfully loaded the serialized machine learning model, accepted user input, generated predictions, and displayed results through the web interface.

Project documentation was finalized by organizing screenshots, architecture diagrams, implementation details, and testing results into a structured report. The completed documentation accurately represents the development process followed throughout the four-week project.

Figure 25. Final version of the phishing website detection application.



The successful completion of testing, interface refinement, optimization, and documentation marked the completion of the phishing website detection system. The developed application provides a practical demonstration of integrating machine learning with a Flask-based web application for real-time phishing URL classification.

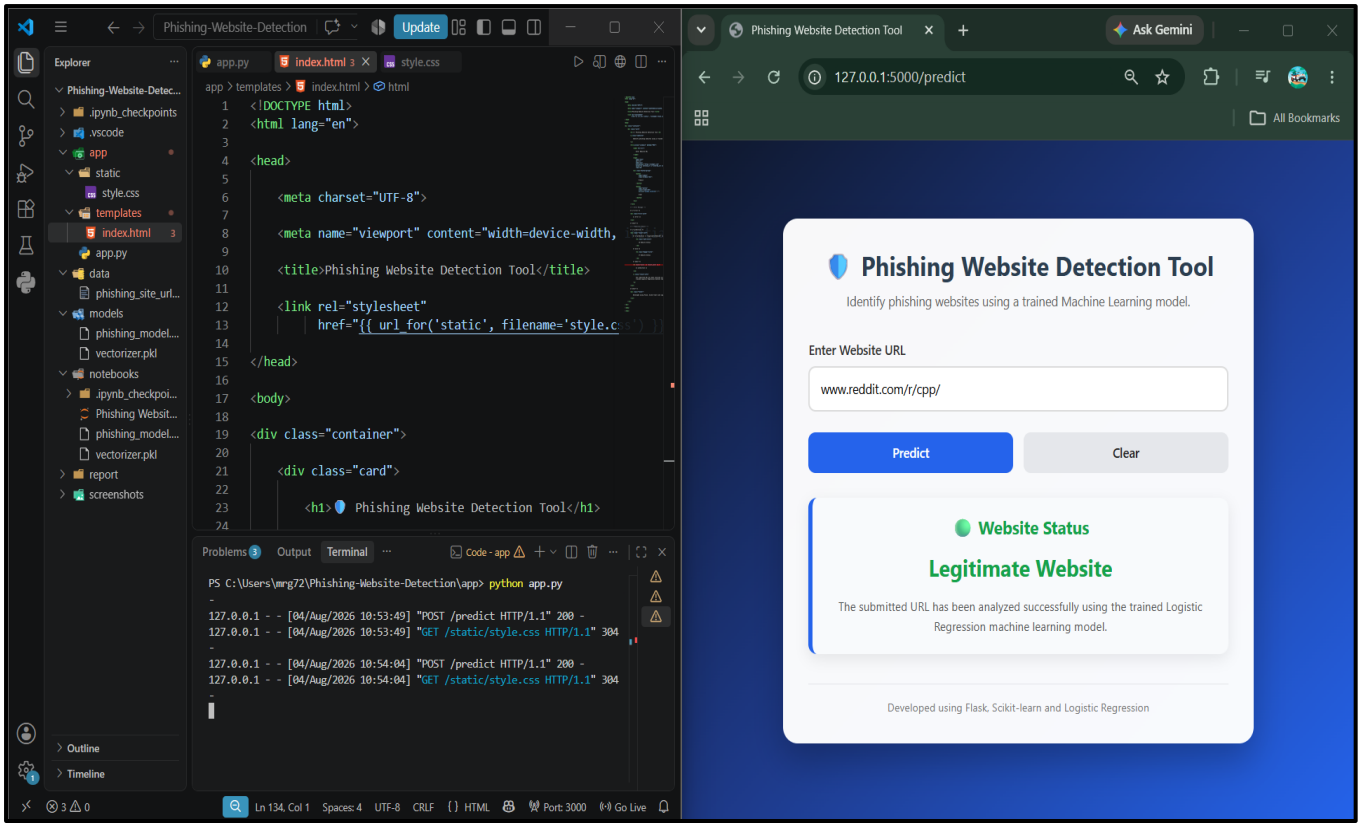
6.3 Final Testing and Validation

The completed application was validated using multiple legitimate and phishing URLs to evaluate its end-to-end functionality. The testing verified that:

- The Flask application starts successfully without runtime errors.
- The trained Logistic Regression model and CountVectorizer are loaded correctly.
- User-entered URLs are successfully transformed into numerical feature vectors.
- The machine learning model correctly generates phishing or legitimate predictions.
- Prediction results are displayed through the web interface.
- Invalid or empty inputs are handled appropriately.
- The application performs consistently during repeated testing.

The testing confirmed that all major functional requirements defined at the beginning of the project were successfully achieved.

Figure 26. End-to-end testing and validation of the phishing website detection system.



6.4 Skills and Knowledge Acquired

During the fourth week, the following concepts and practical skills were developed:

- Functional testing of machine learning web applications.
- User interface enhancement using HTML and CSS.
- Input validation and basic error handling.
- Debugging and application optimization.
- End-to-end integration testing.
- Software documentation and report preparation.
- Deployment readiness of machine learning applications.
- Practical understanding of integrating machine learning models with Flask for real-time prediction.

6.5 Project Outcome

The phishing website detection system was successfully developed as a Flask-based web application for real-time URL classification. The project covered the complete machine learning workflow, including dataset preprocessing, feature extraction, model training, evaluation, deployment, and testing. The application enables users to submit website URLs and receive phishing predictions using a trained Logistic Regression model. While based on lexical URL analysis, it demonstrates the practical application of machine learning in cybersecurity and provides a foundation for future enhancements such as webpage content analysis, domain reputation, and SSL certificate verification.

CHAPTER 7: CONCLUSION

7.1 Conclusion

This project successfully developed a **Phishing Website Detection Tool Using Machine Learning** capable of classifying website URLs as either legitimate or phishing. The project followed a systematic development process beginning with dataset collection and preprocessing, followed by exploratory data analysis, feature extraction, machine learning model training, evaluation, deployment, and application testing.

Multiple machine learning algorithms were evaluated using standard performance metrics such as Accuracy, Precision, Recall, and F1-Score. Among the evaluated classifiers, the Logistic Regression model achieved the best overall performance and was selected for deployment. The trained model and CountVectorizer were serialized using the Joblib library and successfully integrated into a Flask-based web application.

The developed application provides an intuitive web interface through which users can enter website URLs and receive real-time phishing predictions. Functional testing confirmed that the application correctly loads the trained model, processes user input, performs predictions, and displays the classification results without runtime errors.

Overall, the project demonstrates the practical application of machine learning techniques in cybersecurity and provides an effective educational prototype for phishing website detection using lexical URL analysis.

7.2 Future Scope

Although the developed system successfully performs phishing URL classification, several improvements can further enhance its performance and capabilities.

Future enhancements may include:

- Integration of deep learning models such as LSTM or Transformer-based architectures for improved phishing detection.
- Incorporation of webpage content analysis in addition to lexical URL analysis.
- Verification of SSL certificates and domain reputation during prediction.
- Integration with browser extensions for real-time phishing protection.
- Development of a REST API to allow integration with other cybersecurity applications.
- Deployment of the application on cloud platforms for public accessibility.
- Continuous retraining of the model using newly collected phishing datasets to improve detection of emerging phishing attacks.

REFERENCES

[1] Tarun Tiwari, *Phishing Site URLs Dataset*, Kaggle.
Available: https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] Python Software Foundation, *Python 3 Documentation*.
Available: https://docs.python.org/3/

[4] Flask Documentation, *Flask Web Development Framework*.
Available: https://flask.palletsprojects.com/

[5] C. C. Aggarwal, *Machine Learning for Text*. Cham, Switzerland: Springer, 2018.

[6] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill Education, 1997.

[7] PhishTank, *PhishTank: A Collaborative Clearing House for Data and Information About Phishing*. Available: https://phishtank.org/

[8] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Burlington, MA, USA: Morgan Kaufmann, 2011.

[9] Scikit-learn Developers, *CountVectorizer Documentation*.
Available:
https://scikitlearn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

[10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[11] C. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[12] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Education, 2009.